Scale — Commitments Ledger

The initial design of Scale makes a set of **cell-semantics** commitments — claims about what the language is supposed to be. This ledger mirrors those commitments and marks each one's current status, so that "we are not doing this yet" is an explicit, reviewable act rather than a silent omission.

This document is deliberately separate from the *technical-debt register* (`docs/technical-debt-register.typ`). The register tracks **host-encoding** debt (where the F# implementation leaks host types into core). This ledger tracks **cell-semantics** commitments. Conflating the two levels is what previously allowed several commitments to fall through unnoticed.

Status values: **Met** · **Partial** · **Broken** · **Absent** · **Deferred (named)** · **Stub**

Commitment	Status	Statement	Depends on
Tensor is the only semantic type	**Met (declared)**	Governs cell semantics only; host types are encoding (see `types.fs` level model).	—
Tensor algebra beyond scalars (W11/D60)	**Met (D60)**	`add`/`subtract`/`multiply`/`divide` and the comparisons are elementwise over two tensors of the same shape; a scalar is the rank-0 case. A shape mismatch is a named `ShapeMismatch`, never a silent first-element read or a broadcast. `ScalarRead.readScalar` refuses a non-scalar.	Tensor
`nor` is the sole primitive of Vivid	**Met (Tier 1, property-pinned)**	`nor` is `U A B nor` = `U \ (A ∪ B)` over an explicit, declared universe model — no category error, no history-dependence. The default-universe form is the unary `complement` over a declared sort (W3/D14); the ternary `nor` keeps its arity because a binary form is ambiguous on a stack. Properties: `nor` is elementwise set complement over random universes; `and`/`or` agree with ∩/∪ over random models.	Sorts
Sorts are `(option Name, Extension)` (D12)	**Met (W3, property-pinned)**	A sort is a pair: a nominal seal (an image text entry) and an extension (member identities). `[ … ]` builds anonymous sets, `:Name` references a seal, `S :Name seal` names one; `same-shape` reads the extension, `same-name` reads the seal, `union`/`intersect`/`except`/`subset`/`member` are structural, `merge` is seal-preserving, and `complement` reads a declared domain. Declarations live in an image-resident root; latest-write-wins; domains only grow. Properties: union/intersect/except agree with set algebra.	—

Commitment	Status	Statement	Depends on
Unification as a pillar	**Met (W4/D15, D59; property-pinned)**	First-order structural unification over image-resident terms: variables (`?name`), atoms, compounds (`-2` tagged), and typed terms (`-3` tagged). `walk` resolves substitutions, `occursIn` rejects direct and indirect cycles, aliases canonicalise to the lower `EntryIdentity`, compounds unify head-and-arguments, and typed terms unify only under the same sort **name** (nominal, D12/D15). A failed match is transactional; `lookup`/`resolve` apply the substitution through chains; `term`/`typed` build terms. Properties: a random variable chain resolves to its tail value; an occurs-check rejects a variable nested at random depth. D59 adds the Miller-pattern higher-order fragment: lambda terms (`-15`), beta-reduction, pattern binding, alpha-renaming, and a three-way `UnificationVerdictTag = -27`; non-pattern spines are `Undecided`, never a false.	Value typing
Multiple Dispatch as the only polymorphism	**Met (W10, D19, property-pinned)**	Generic functions and methods are image-resident metaobjects (`-4`/`-5`) under `GenericFunctionsRoot`, latest-write-wins by name. Selection is type-first: a typed operand uses its seal, an untyped operand infers the unique most-specific declared sort containing it, and no unique type is `UnknownType`. An exact specializer tuple is the static fast path; the dynamic fallback is extension inclusion (known sort) or membership (`UnknownType`); specificity is extension inclusion. Incomparable or extension-equal maximal methods are `AmbiguousDispatch`; none is `NoApplicableMethod`; neither is a silent pick. A method body receives the underlying values. `executeWord` remains the lexical layer. Property: dispatch selects exactly the declared sort members.	Value typing
Metaobject Protocol as open implementation (D16, D17)	**Partial (D17 stages applied; kernel reduction continues: D19 methods, D21 word metaobjects, D31 predicate**	The language's extension points — words, sorts, generic functions and methods, guard evaluators, protocol failures — become image-resident, tensor-encoded metaobjects with a protocol and a fixed meta-level. The bootstrap kernel is explicit and inventoried (`docs/kernel-inventory.typ`); the goal is	Value sets (W3), W4 (D15), W10 (dispatch), W8 (D29)

Commitment	Status	Statement	Depends on
	sorts, D32 classification, D35 guards, D36 sort constructors, D39 set-operation methods, D41 guard-lattice bridge, D43–D51 cell and control layers)	kernel reduction, not a reflective tower. Generic functions and methods are image-resident (`-4`/`-5`) and a new method needs no F# (D19); word metaobjects are image-resident (D21); a predicate sort carries its membership quotation and answers `Member`/`NotMember`/`Undecided` (D31); `class-of` is one total verdict over entry kind and the unique most-specific sort, and dispatch uses the same membership verdict (D32); guard metaobjects are image-resident (D35); and a named sort constructor is image-resident and a new constructor needs no F# (D36). A document refinement can name an image-resident guard, and the verification lattice evaluates it through the evaluator's resolver (D41). The full cell system — classes, slots, instances, class precedence, method combination, closures, and metaclasses — landed as D44–D50; a reflective evaluator remains out of scope.	
Guard evaluation is one protocol (D35, D41)	**Met (runtime half D35, lattice half D41)**	A guard is an image-resident metaobject (D35); a document refinement may name one (`<field> guard <name>`), and the verification lattice resolves it through `EvaluatorRuntimeGuard.resolver`. A guard verdict is `GuardHolds`/`GuardDoesNotHold`/`GuardUndecided`, so an undecidable guard is never a silent false. A named guard is executable and generative, but a quotation has no SMT encoding: Z3 reports `Unavailable` and the emitter refuses it by name, never approximating it. The typed-only lattice path also refuses it. The runtime and the refiner therefore cannot disagree about a field value.	W3 (value sets), D35 (runtime guard), D25 (lattice)
Event-Stream as the program	**Met (W8, D29; property- and example-pinned)**	Every executed token appends an image-resident, append-only step (`StepTag = -10.0`) carrying the program-counter metadata, the token, the stack before it ran, and the program it resumes. The history root is a vector of step addresses; rollback restores a step and re-points the root at the steps before it without rewriting the image. `history` reports the steps and `rollback` restores one; bookmarks are absorbed as	Error semantics, W5

Commitment	Status	Statement	Depends on
		degenerate one-point events, with `jump` restoring through the same rollback. The history covers the top-level `run` loop; a set literal's inner interpreter and a `guard` probe are expressions, not steps.	
Linear ownership per run (W9, D30)	**Met (W9, example-pinned)**	Ownership is execution state (D1): a run maps each value address to its owning actor and tracks consumed addresses. `own` takes ownership, `borrow` is a checked read, `consume` ends the value's life for the run, and `release` gives ownership up; `NotOwned`, `OwnedByAnother`, and `ValueConsumed` are named refusals. A spawned actor starts owning nothing, and the image is never rewritten: the checks are dynamic and advisory at the ownership words, not a static type system. Ownership is never serialized.	Error semantics, W8
ChannelEvents	**Met (W7, D26; state hardened in D27)**	Events are image-resident tensors (`EventTag = -8.0`): `send-event`/`receive-event` construct channel operations, `choose`/`wrap`/`guard`/`with-fallback` compose them, and `sync` commits one. A would-block registers the actor as an event waiter and halts it; `send`/`close` wakes it to retry. `wrap` prepends its quotation after the commit, `guard` evaluates a discarded probe, and unselected `with-fallback` branches commit their ready nacks. Blocking is not an `Error`. A channel's own state — its open flag and its queued messages — is image-resident and append-only (D27): the identity entry is written once, every state change appends a state entry, a queued message survives persistence, and a terminated actor leaves the waiter lists so it cannot swallow a message.	Channels (W6)
Erlang-equivalent preemptive scheduling	**Met (W5, D63)**	A cooperative scheduler lands: per-process state threaded through `EvaluationState`, `spawn`/`yield`/`halt`/`resume`/`kill`, and budget exhaustion as a yield. D63 adds the preemption that was deferred: an actor carries a `Consumed` token count and an optional `Deadline`; reaching it is a `TimeoutExit`, and a supervisor (`one-for-one`/`one-for-all`/`rest-for-one`,	Error semantics, event-stream

Commitment	Status	Statement	Depends on
		`supervise`) restarts the affected children from their `ActorSpec` under a bounded restart budget. A blocked actor consumes nothing, so a deadline never fires on it. Wall-clock deadlines, priorities, and mailbox limits remain non-goals.	
Actor addresses are private capabilities (D64)	**Met (D64)**	`spawn`/`supervise` push an image-resident handle (`ActorHandleTag = -28.0`); `kill`/`resume` accept only a registered live handle and refuse a number, a forged entry, a foreign entry, or a retired one with `NotAnActorHandle`. `actors` reports statuses only. The map is execution state and is never serialized; a restart takes a fresh identity and a fresh handle. The holder of a handle can target the actor, like a PID; a self handle exists, but `kill` refuses self.	Scheduler (W5), D63
Actor mailboxes, self, and links (D65)	**Met (D65)**	Every actor owns a private mailbox (a hidden channel) and a private handle. `self` pushes the handle; `send-to` queues in the target's mailbox; `receive-message` dequeues or blocks like `receive`. `link`/`unlink` maintain a mutual link; an abnormal exit delivers a pending exit to every live linked actor, consumed before its next token, while a normal exit signals nobody. Links are per incarnation. The mailbox id never reaches the stack. Mailboxes are unbounded.	Scheduler (W5), Channels (W6), D63, D64
Trapped link exits (D66)	**Met (D66)**	`trap-exit` makes a link's abnormal exit an `[ExitSignalTag; reasonCode]` mailbox message instead of a kill; `exit-signal?` and `exit-reason` inspect it. The signal reuses the mailbox delivery path, so a blocked receiver wakes with it. Non-trapping actors keep the D65 kill, and normal exits signal nobody. Selective receive remained a non-goal at D66; D67 adds monitors and the registry, and D68 adds selective receive.	Scheduler (W5), D65
Monitors and the named registry (D67)	**Met (D67)**	`monitor` returns a per-run reference; on any exit of the target a `[DownTag; ref; reasonCode]` message reaches the watcher once and the monitor is consumed. `demonitor` removes it, and a forged or foreign ref is `NotAMonitor`. `register`/`unregister` map a text name to an actor;	Scheduler (W5), D65, D66

Commitment	Status	Statement	Depends on
		`whereis` returns its handle; a live name is exclusive and a terminated actor's names are removed. Both maps are execution state. Selective receive remained the one actor-model operation absent at D67; D68 adds it, propagates a supervisor's limit error to its own supervisor, fires monitors on cascading kills, and unifies the `send` target.	
The actor boundaries (D68)	**Met (D68)**	`receive-matching` scans a mailbox oldest-first with a predicate quotation, leaves non-matching messages queued in order, and blocks to rescan on a later send; `SelectiveWaiters` is execution state and is never serialized. `send`/`send-event` accept an actor handle or a channel, while `receive`/`close` remain channel-only. A supervisor that spends its restart budget exits with the limit error to its own supervisor instead of aborting the run. A cascading kill fires the monitors of every terminated descendant. Bounded mailboxes, priorities, and selective-receive timeouts remain non-goals; implementation-instance IPC is a later target (D151).	Scheduler (W5), D65, D66, D67
One actor-word guard (D69)	**Met (D69)**	A quotation run and a set literal use the same `EvaluatorQuotationRun.actorWords` list and pre-scan the whole body, so a nested quotation or `apply` cannot smuggle an actor word past the guard. The list covers the mailbox, link, monitor, registry, send, close, and selective-receive words added by D65–D68. Read-only actor words (`self`, `whereis`, `down?`, `exit-signal?`) stay available; event constructors stay available until `sync` commits. The state boundary also resets `SelectiveWaiters`, and a killed selective waiter is removed.	Scheduler (W5), D65, D66, D67, D68
Reserved actor system messages (D75)	**Met (D75)**	Every actor has a runtime-reserved message class that is checked before user delivery. `SystemMessageTag` covers Suspend, Resume, Code-Change, Status, and Shutdown; a system message never enters the mailbox queue. `Suspended` is execution state and is never serialized. Status replies use `SystemStatusTag`; self-targeted suspend/resume/code-change/shutdown are	Scheduler (W5), D63, D64, D65, D66, D67, D68, D69

Commitment	Status	Statement	Depends on
		refused. No MOP, mailbox-structure, or linear-region change. Shutdown terminates the target subtree with `ShutdownExit` and fires the terminated actors' links and monitors.	
Discourse contradiction detector (D77)	**Met (D77)**	A direct syntactic contradiction over one claim: same subject, ordered conditions, and verb, with Every/No or Some/No. `Discourse.contradiction` returns the first committed statement the candidate contradicts as a `Layer.Discourse Contradiction` violation. The showcase renders it. It is not an SMT consistency check and does not handle paraphrases, reordered conditions, or transitions.	Diagnostics (D57, D70–D72, D76)
SMT violation in the showcase sink (D78)	**Met (D78)**	The showcase creates the demo encoding, discharges an empty-condition `Capability`, converts the falsified result with `Violations.ofSmtDischarge`, and adds it with `Diagnostics.addOption`. It uses the same sink and presentation path as the parser, table, and discourse examples. This is showcase wiring, not a CLI SMT mode or a change to the acceptance gate.	Diagnostics (D62, D74, D77)
Typed proposals and advice (D79)	**Met (D79)**	The mixed `Patch` with an optional specification edit is gone. `ImplementationProposal` cannot carry a spec edit; `FrozenSpecification` carries a private `SpecificationIdentity`; `SpecificationProposal` requires a private exploratory capability; `Violation.Suggestion` is advice, not a proposal. The `FixedSpecificationAcceptance` gate consumes a frozen specification and an implementation proposal and never calls `NearestSentence`.	D11, D30, D71, D76, D77
Scale ⟶ F# ⟶ SMT (Z3) (Hedgehog/ Idefix; Lean as optional export)	**Partial (W1, W2, W12, W13, W14, W15 done; harness wired; refinement loop on compiled output; SMT discharge, proved refiner closure, and fixed-spec**	One emitter (`emitter.fs`) renders every artifact, parameterized by a domain binding supplied as data (D9). It produces framework-free, known-compiling property predicates (D5), pinned by a golden test; the parser enforces the Prose sentence grammar, a closed sentence grammar over vocabulary categories, and refuses unsupported constructs by name (D8, D18);	—

Commitment	Status	Statement	Depends on
	gate applied, D10/ D11)	the Hedgehog 1.0.0 property harness over the kernel lives in `properties.fs` (34 properties, run in the suite); and the refinement loop — specification, compiled property, falsification, synthesized refinement, compiled confirmation — runs end to end in `components/demo/demo/` (D7, D9). The loop executes the emitter's *compiled* output: `components/demo/demo/generated/PlayerSpecificationPristine.fs` and `PlayerSpecificationRefined.fs` are checked in, compiled into `bases/scale-cli/scale.fsproj`, invoked by a thin harness, and pinned byte-identical to a re-emission (D9 applied). The D18 transition surface landed as D55 (`when <subject> <condition>, every <subject> <verb>.` projects to `Transition`). D56 closes the large-domain remainder: a symbolic lattice seeds guards from the solver's counterexample, widens and bisects to the weakest proved threshold, and reports `Coverage = 0` because coverage is not measurable there. **D33 later made existential `some` executable: it projects to `Possibility`, rendering a witness predicate and a satisfiability obligation (`Witnessed`/`NoWitness`), never a universal implication. W13 applied (D10): the implementation is one `DomainExpression` body with an interpreter for Hedgehog and an encoder for Z3; the SMT encoding is generated from the same parsed document, discharged with `Proved`/ `Falsified (model)`/`Unknown`/`Unavailable` outcomes, validated differentially against the compiled F# predicate, and the `.smt2` obligation is byte-pinned. The `.lean` stub and `LeanBinding` are retired. W14 applies the proved refiner closure: the refiner discharges `pre ∧ guard ∧ ¬post`, keeps the universe only for candidate thresholds and coverage, and records the solver version and witness model. W15 applies fixed-spec**	

Commitment	Status	Statement	Depends on
		mode (D11): `repair.fs` holds the spec fixed, turns a witness into a versioned repair bundle, and accepts a patch only when the target proves, the checked-in corpus does not regress, the spec hash is unchanged, and the encoding agrees with the compiled predicate.	
Verification coordinates shared by Z3, Hedgehog, and the refiner (D25, W16, D91)	**Met (W16, D91)**	One verification lattice describes each field's name, sort, finite coordinates, and narrowing. Every numeric coordinate and guard threshold is checked against the encoded sort's boundary — Int32 for Integer and Natural, a finite double for Real — so Hedgehog cannot sample a value Z3 cannot represent. D91 adds the Real and Natural sorts, sort-aware emitter threshold validation, and a sort-aware symbolic refiner. The record representations are private; `create`, `ofFieldTypes`, `applyGuard`, and `withGuards` are the public constructors. `Refiner.refine` folds the document's existing refinements into coverage, and the emitter refuses a lattice narrowing the document does not carry. The pinned artifacts do not move. D56 adds the witness-guided path: a lattice with no finite coordinates is narrowed from the solver's counterexample, with the same `pre ∧ guard ∧ ¬post` proof gate and `Coverage = 0` because coverage is not measurable there.	Sorts (D12), W14, W15, D56, D91
Literal specification sentences (Prose sentence grammar, W2.1/D18)	**Partial (Stage 1 + existential `some` + transitions + disjunction)**	The sentence language is an explicit grammar: sentence-initial `every`/`no`/`some`, TypeName subject, optional `that` relative clause of conditions joined by `and` and `or` (`and` binds tighter, D58), FunctionName verb, required full stop. `every` projects to `Capability`; `no` projects to `Invariant` and renders a negated postcondition; `some` projects to `Possibility` (D33) and renders a witness predicate with a satisfiability obligation. A boolean condition is a predicate phrase whose surface form begins with `is` (canonicalized, for example `isAlive`) or explicit `is <field>`; `has <field> <comparison> <integer>` is executable end-to-end as a typed numeric	Tokenizer, parser (D8); emitter (D5)

Commitment	Status	Statement	Depends on
		condition (D24). The event-triggered shape `when <subject> <condition>, every <subject> <verb>.` projects to `Transition` (D55): the trigger's conditions are the precondition, the verb is the effect, and the F# artifact is a pinned guarded effect. A disjunction is one DNF atom over atomic conjunctions (D58): the emitter renders `||` over `&&`, the SMT obligation is `MkOr` over `MkAnd`, and the canonical text round-trips without a corpus-schema change. Articles, pronouns, `when` away from a transition's start, `not`/`never`, `all`, unknown words, and malformed sentences are refused by named `ParseError` cases, as are a mismatched transition subject, a non-universal outcome, and a missing transition comma. Pinned by the suite's parser examples and one generated-sentence property that includes numeric `has`.	
Violation reporting (D57/D61/ D62/D70/ D71/D72)	**Met (P0–P5, library-level)**	Every failure is one `Violation`: layer, rule, span, expected, given, blame (`Input` vs `CoverageGap`), witness, and repair. P0 maps every `Parser.ParseError` case and returns `Result<Sentence, Violation>`; the span is the offending canonical token, not a character offset. P1 maps every `EvaluationError` case and every guard verdict, with undecidable membership, dispatch, unification, guard, or budget reported as `CoverageGap`. P2 maps every SMT outcome: a falsified obligation carries the solver model, a proved obligation or a refuted possibility carries the labelled unsat core, and `Unknown`/`Unavailable` stay `CoverageGap`. P3 is applied as D70: `Discourse.utter` publishes the violation and rolls back an `Input` fault at `Structural` or above, while a `CoverageGap` does not block. P4 is applied as D71: `NearestSentence.candidates` finds the nearest accepted token list under the fixed grammar, and `candidatesWith` ranks candidates through a layer check. P5 is applied as D72: `Presentation.render` gives the layer-aware prose, expected/given,	Parser (D18), evaluator, D70, D71, D72

Commitment	Status	Statement	Depends on
		span, evidence, and suggestion; `renderTechnical` keeps the rule id. These APIs are library-level and exercised by the suite; the CLI and demo consume them. `NearestSentence` produces advice; `FixedSpecificationAcceptance` is the fixed-spec implementation gate and never calls the sentence search. The `Discourse` layer has the D77 contradiction detector; `Discourse.utter` is the rollback policy. D76 adds source offsets, D78 puts the SMT violation in the showcase sink, and D79 separates typed proposals from advice. See `docs/prose-foundations.typ` and `docs/repair-model-analysis.typ`.	
Decision tables as a specification refinement (D13; D73; D74)	**Met (D73, D74)**	A symbolic table of guarded rows (`Table<'outcome>`) with `check` proving completeness ($\neg(g_1 \vee \ldots \vee g_n)$ unsatisfiable) and, under `Unique`, disjointness ($g_i \wedge g_j$ unsatisfiable); each failure carries a `Smt.Model` witness. `select` is pure over `Refiner.evaluateGuard` (`Selected`/`NoMatch`/`Ambiguous`). The promotion adds `TableBinding` (one body per outcome), `obligationFor`/`obligations`, and `DecisionTableEmitter.emit`: one emitted property per row with the row's guards as the precondition and `rowN` as the body the host supplies. D74 adds the Prose sentence table surface (`table`, optional `policy:`, transition rows, `end`) and `Diagnostics.Sink`; `DecisionTable.explain`/ `addSelection` feed the sink, and `render` uses `Presentation`. The CLI table mode is `--table <path>`; `--diagnose <sentence>` wires parser diagnostics, repair and presentation. There is no new sentence grammar, and a domain that does not declare a table is unchanged. Machine-learning decision models remain outside the proof-or-witness trust boundary.	D12 (sorts for `Inputs`), D10 (the solver), D73, D74
DSPy-shaped model-backed	**Prepared (optional)**	A native signature is a projection of the `.scale` contract and the verification lattice, not a second source of truth; one model-touching `predict` word is an edge	D9, D11, D25, D41

Commitment	Status	Statement	Depends on
programs (D42)		effect returning named refusals; a host-side optimizer searches instruction text and examples only and appends a new artifact; the fast metric is search-only and the slow metric is the existing `Smt.discharge` + `Repair.accept`. Model calls are recorded for replay. Constrained decoding is graded: only structural or finite signatures drive a grammar; predicate sorts are post-parse and may be `Undecided`. No implementation, dependency, or word exists today; the core is unchanged.	
The full cell system (D43–D50)	**Met (D44 encoding, D45 closures, D46 classes/slots/ instances, D47 precedence/ dispatch, D48 method combination, D49 metaclasses, D50 conditions/ restarts)**	A class is an image-resident metaobject with a name, superclasses, slots, a metaclass, and a method-combination policy; a slot is a definition with a name, a value set, an optional default, and an initialization rule; an instance has stable address identity and an appended state (like channel state, D27); class precedence is deterministic; class dispatch is nominal; methods gain a qualifier and a method combination orders them; closures are image-resident code cells with an environment chain; metaclasses close at a root. The surface stays the Prose sentence grammar; the entry shapes are internal encodings, never syntax. The layer uses runtime value sets, not the verification `FieldType`. The encoding landed as D44: tags, codecs, the class and instance-state roots, and persistence. Closures and lexical scope landed as D45: `let` creates a frame, `lambda` captures it, `apply` runs in it and restores the caller, and `lookup` reads the frame chain before the global table. Classes, slots, and instances landed as D46: `class`, `slot`, `create-instance`, `slot-value`, and `set-slot-value`; an instance has stable identity and appended state. Class precedence is C3 and class specializers participate in generic dispatch (D47). Method combination, metaclasses, and conditions/restarts landed as D48–D50. No dependency changed.	D16, D17, D27, D31, D32, D43, D44, D45, D46, D47, D48, D49, D50

Commitment	Status	Statement	Depends on
Error semantics as values (Railway)	**Met (property-pinned; D27)**	`executeWord` returns `Result`; `run` attaches position and returns a `Diagnostic`. Defect vs. non-event kept distinct. Property: 100 random programs per run yield `Ok`/`Error Diagnostic`, never an exception — over the whole word surface, not only the arithmetic core (D27). An inconclusive outcome is not a pass: a Hedgehog run that gave up is reported as `Inconclusive`, and the demo says so.	—
Image reference is a nominal `EntryIdentity`	**Met (Wave 0)**	`Lexicon`/`Image`/`Stack`/`TextCache` keyed by `[<Struct>] EntryIdentity`; sentinel replaced by `EntryIdentity option`. Host encoding with a tensor projection.	—
Environments are image-resident (D2)	**Met (property-pinned)**	`UserWords`/`Bindings`/`Models` live in the image as association vectors behind roots (`UserWordsRoot`/`BindingsRoot`/`SetsRoot`); latest-write-wins lookup; binding *data* survives persistence, with the root pointer re-threaded by the caller (execution state per D4). Properties: latest-write-wins over random rebinds; persistence round-trip with re-threaded root.	—
Execution state split by lifetime	**Met**	Image holds all data and definitions; `Program` and PC counters are control state (D4), so nothing outside the image is data.	—
One execution-root registry (R5)	**Met (D191)**	`RootKind`/`rootKinds` is the single enumeration; `getRoot`/`setRoot` reattach a root; the reflection coverage test fails for a new case. Roots remain execution state and are reattached after a store round-trip.	—
Vocabulary has one source of truth (Tier 2)	**Met (property-pinned)**	Edge modules derive the vocabulary list from the image (`readVocabulary`); `Tokenizer`/`Parser` respect `IsActive`, so edge and core agree on word availability (regression tests). Property: generated sentences never tokenize into inactive tokens.	—

Dependency structure

```
flowchart TD
    ERR["Error semantics: defect vs event"] --> SUP["Supervisor seam"]
    ES["Event-stream: history + PC metadata"] --> SUP
    ES --> RB["Rollback"]
    SUP --> SCH["Scheduler replacement: landed (W5, per-process state)"]
    RC["Reductions counter: keep as-is"] --> SCH
    SCH --> PRE["Preemptive scheduling: landed (W5, D63)"]
```

```
    SCH --> CH["Channels"]
    CH --> ChannelEvents["ChannelEvents: landed (W7, D26)"]
    VT["Value typing: sorts / ValueTag<br/>a sort is (Name, Extension) — D12/W3
landed"] --> MD["Multiple Dispatch<br/>dispatches on the extension"]
    MD --> META["Metaobject Protocol: open implementation<br/>words, sorts, methods;
fixed meta-level (D16/D17)"]
    VT --> UN["Unification: occurs-check, bidirectional, resolution<br/>unifies under
the name — W4/D15 landed"]
    VT --> NOR["nor universe: ternary unchanged<br/>complement defaults to a declared
sort"]
    VT --> DT["Decision tables: rows + completeness (D13, optional)"]
    DT -. optional .-> PIPE
    PAR["Parser: Prose sentence grammar (D18); Emitter: W1, W2.1, W12 done (D5, D8,
D9)"] --> PIPE["Scale → F# → SMT (Z3) (harness wired; loop runs compiled output, D7/
D9; SMT discharge and proved refiner closure applied, D10/W13-W14; fixed-spec gate
applied, D11/W15)"]
```

Two chains plus one isolated track: a **concurrency chain** (errors → supervisor → scheduler → preemption → channels → ChannelEvents), a **typing chain** (value typing → multiple dispatch → Metaobject Protocol, unification, `nor`), and the **pipeline track** (independent). The dashed edge is the optional decision-table refinement (D13): it reads sorts from the typing chain and would feed per-row obligations to the pipeline, but nothing depends on it today. The Metaobject Protocol is a separate track (D16/D17), downstream of multiple dispatch; it is landed as D44–D50, and its baseline is `docs/kernel-inventory.typ`.

Notes

- **Multiple Dispatch** is now landed (W10, D19), so it is not merely single-dispatch: `executeWord` remains the lexical layer, and a token that is neither a built-in, a user word, nor a generic function falls through to the vocabulary. A generic function dispatches on operand sorts, not on the token string.
- **Bookmarks are absorbed by the event-stream (D29).** A bookmark now records a step and names it; `jump` restores through the event-stream's rollback. The bookmark surface remains as the degenerate one-point view.
- **Unification and `nor` are co-design.** Both need a typed-term representation and a resolution step (substitution vs. sort-membership), so they share the `ValueTag` substrate — which D12's W3 registry now realizes as the sort's **nominal** component (the name `N`), kept image-resident so the pairing is not host state. W4/D15 lands the substitution half: the sorting wrapper is a `-3` tagged term, and unification compares `N`, not the extension.
- **Structural and nominal are one record, not two systems (D12).** A sort is `(N : option Name, X ⊆ U)`; nominal equality reads `N`, structural equality reads `X` (extensionality). The set operations are the dial between them, and the value widens past both to multiset and sequence. See `docs/set-theoretic-typing.typ`.
- **Channels exist in the tree.** They landed as W6 (D20) and their state was hardened in D27: the endpoint is an image entry, its open flag and queued messages are appended image state, and only the waiters — who is blocked — stay host-side, as control state.

Memory vocabulary (D80). The image is an append-only value authority, not a heap. Actor-local ephemeral values belong in a per-incarnation epoch; handles and metadata are structs; views are byref-like and transient; `Tensor` is not a `ref struct`. Stage 0 changes no runtime behavior and does not add a per-process GC. The Elixir/BEAM model is in `docs/memory-model.typ`; the Go path is delayed.

Usage modes and trace types (D81). Linearity is a mode on the type: `ExactlyOnce` for a linear resource, `AtMostOnce` for an affine reference, `AnyNumberOfTimes` for an immutable image value. A linear type is a named set of traces; the end-of-trace rule is the mode's obligation. The `linearity` word keeps the linear default. Epoch labels, automatic close, and unmanaged arenas remain later stages.

Reclamation roots (D82). Reclamation reads every non-terminated actor's suspended state, not just the running actor. A terminated actor is not a root. Compaction therefore reclaims entries no live actor holds, while staying conservative and opt-in.

Epoch ranges (D83). An epoch is a set of allocation ranges in the shared frontier. Promotion reclassifies an entry as durable without moving it; close names the unpromoted entries of a closed epoch, and reachability still decides what is freed. The module is pure and is not yet wired into the scheduler.

Epoch recording (D85). The actor incarnation's epoch lives in `OwnershipState`. The scheduler records the outgoing turn's frontier growth, resumes the incoming epoch at the current frontier, and closes the epoch on termination. Promotion at send/persist and physical reclamation remain later stages.

Promotion at send (D86). `send`, `send-to`, `send-event`, and `sync` promote the escaping value in the sender's epoch; the address is unchanged. Persistence-time epoch filtering remains a later host decision.

Host-level persistence (D87). `Persistence.toBytesForState` and `writeToFileForState` serialize only durable or promoted entries across the live actor and every actor snapshot. The frontier and surviving addresses are unchanged, so the result is a sparse image.

Physical reclamation (D88). `Reclaim.compactClosedEpochs` removes a closed epoch's unpromoted entries when no live root reaches them. The scheduler runs it at yield, halt, and kill; addresses and the frontier are unchanged.

Arena contract (D89). `TensorHandle` is a stable storable reference; `TensorView` is a stack-only span projection; the managed arena allocates, resolves by generation, and invalidates every handle on close. The evaluator is not yet routed through it.

Memory freeze (D90). The image/Tensor representation, managed backing, and shared-immutable-with-promotion policy are frozen; copy-on-send and evaluator arena routing are rejected for the port. P1 is unblocked.

Real and Natural sorts (D91). The verification encoding carries Real, Integer, and Natural field sorts. Real is SMT Real and QF_LRA, Integer and Natural are Int and QF_LIA, and a mixed domain is QF_LIRA; Natural is constrained to `>= 0`. The emitter validates a guard threshold against the field sort, and the symbolic refiner uses a Real tolerance and clamps Natural at zero. The integer artifacts are byte-identical.

Unified Julia/Elixir language (D93; revoked by D151). The unified Julia/Elixir language is dropped. Scale's core language is its own concatenative surface, with the Prose sentence grammar and the prose surface as front ends. Different implementations are separate instances in their host languages and integrate through the actor model's IPC, planned by D151. Documentation only; no source or test change.

Actor IPC between instances (D151). Each implementation keeps its host language; actors are the only integration unit. The local actor model, the image promotion rule, and the `scale_transport_send` ABI seam exist. S1 is landed (D154): one Unix-socket connection delivers a

numeric envelope to the root actor and returns its reply. S2 is landed (D155): capability 1 names a worker, the root monitors it, and the client receives the worker's normal exit. Instance identity, discovery, authentication, partition, and security are planned; `docs/actor-ipc.typ` records the model and the staging.

Actor wire protocol (D152). Scale implements its own actor wire protocol, Cap'n Proto-inspired and in-repo. C++ is the encoding and RPC reference, Rust the validation and view reference, Go the buffer, framing, and concurrency reference; Protobuf and OCapN are secondary. The first scope is the envelope and portable payloads; a schema compiler, RPC pipelining, three-party handoff, and distributed GC wait for a consumer. `docs/binary-format.typ` is the plan.

Schema tooling over the wire (D153). The actor message keeps one frame with a logical envelope/body split. The wire follows Cap'n Proto for the frozen subset; compatibility is claimed only after the reference conformance test. The schema layer takes Protobuf's descriptor set, plugin contract, field numbers, and reserved ranges as build-time metadata mapped to ordinals. `oneof` maps to a union. Unknown fields are preserved only by explicit raw-body policy; `Any` is a typed union plus a registry, and `AnyPointer` is validated and whitelisted. Protobuf wire tags never appear on the Scale wire.

First actor IPC slice (D154). The `ipc` component adds `Wire` and `ActorHost`. The wire is a single-segment Cap'n Proto subset with a root struct, seven data words, one pointer word, and numeric or text bodies. The host binds a Unix socket, delivers a numeric envelope to the root actor's mailbox, runs `receive-message; 1; add`, and returns the reply. `--wire-listen` and `--wire-connect` expose it.

Worker capability and remote supervision (D155). The envelope adds a send/reply/monitor/monitor-ack/down operation. `spawnWorker` creates a non-root worker, assigns capability 1, and monitors it from the root. The client monitors capability 1, sends 21, receives 42, and receives `down normal` when the worker terminates; a missing socket is the named `Unreachable` failure. The change fixes `Scheduler.continueAfter` so the threaded store survives a child's exit when no ready actor remains; the S2 integration test is the regression test. One connection, one worker, no discovery or authentication.

BEAM wire harness (D156). `adapters/elixir` implements the Scale actor wire subset in Elixir, a supervised peer, and a BEAM listener with a real worker process. Both cross-language directions pass: the Elixir peer against the F# listener, and the F# client against the BEAM listener. Eight ExUnit tests pass. This is a wire harness, not a language port; the full BEAM port and a Rustler NIF over the C ABI are deferred until the Prelude is in Scale and a native kernel artifact can be tested.

Language-level remote send (D157). `send-to` recognizes a host-registered `[RemoteHandleTag; instance; capability]` and records a `TransportRequest` in `SchedulerState.PendingTransport`; it performs no I/O and needs no local mailbox. A tagged tensor that the host did not register is not a capability.

Remote reply delivery (D158). `LanguageTransport.deliverPending` drains the transport queue, sends each request, adds the numeric reply to the store, delivers it to the sender's mailbox, and resumes the sender. `send-to; receive-message` returns the remote result. One sender per drain; numeric replies only; the down reason remains a host result.

Wire golden vectors and reference interop (D159). `components/ipc/vectors/` pins six positive vectors and two negative vectors with a Cap'n Proto schema. `capnp encode` generates the reference bytes; the F# test decodes them and re-encodes byte-for-byte; `check.sh` re-encodes every text vector

with `capnp` and decodes the Elixir encoder's bytes. The frozen subset is conformance-tested; far pointers and multi-segment messages remain out.

Typed term IR (D160). The target runtime is `Program: Term list`; a quotation is `Quote of Term list`, and `apply` splices. The kernel owns the core term encoding, the term-list loop, the structural and stack-effect verifier, the `VerifiedTermProgram` receipt, the verify entry, and the ABI/image schema. The Prelude owns surface elaboration, word signatures, language type rules, effects, dynamic-code policy, macros, and lowering. The domain SMT verification stays separate. Planned phases P0–P8 are in `docs/term-ir.typ`.

Term shape and portable document (D161). P1 lands `TermIr` and `TermPortable` in the kernel: the core term variants, `TermProgram`, canonical JSON, and six positive plus two negative vectors under `components/kernel/term-vectors/`. The test decodes each vector and re-encodes it byte-for-byte. No term executes and no stack is typed; P2 adds the verifier and receipt.

Token-to-term boundary (D163). P3a adds `EvaluatorProgram` in the Prelude. `ofTokens` maps a finite numeric token to `LiteralNumber` and every other token to `Word`; a non-finite number is a named refusal. `ofText` splits external text and reuses the same boundary. `toTokens` is the temporary projection for the kernel bridge and accepts only word/number terms. `EvaluationState.Program` is still `string list`; P3b changes the active state, P3c adds tagged term documents for quotations, and P3d removes the projection and text execution. P3b's contract is recorded as D164; its test batch and migration are next.

Typed control state (D165). P3b is implemented. `EvaluationState.Program` and the caller frames on `HandlerFrame`, `MethodFrame`, and `CombinationFrame` are `Term list`; the loop executes `LiteralNumber` and `Word`, and any other active term is an invariant failure. `EvaluatorProgram.toKernelTokens` is the temporary total projection for those two shapes, and `ofTokensOrFail` is the total construction form. The control words, the token-vector history, and the kernel bridge project at their boundary and convert back with `ofTokens`, so text semantics are unchanged. The differential corpus keeps `Case.Program` as `string list` and converts only at `runInterpreter`; `KernelState.Program` and `LoweredAbstraction.Program` remain `string list`. P3c adds tagged quotation values, P3d removes the projection and token-vector history, and P4 adds stack typing.

Quotation code values (D166). P3c is implemented. A quotation is the tagged code value `[QuoteTag; documentId]`; the document is the canonical `term-program` JSON for the body. `EvaluatorCode.encode` creates the value and `EvaluatorCode.decode` returns its terms. `{ ... }` collects nested `Quote` terms, `Quote` execution pushes a code value, and `if`, `apply`, `define`, `control`, `lambda`, closures, actor bodies, handlers, methods, predicates, guards, and set quotations decode code values. History stores code values in its program and current-term fields. A legacy text quotation is still readable during P3c; P3d removes that reader and the remaining token projections.

F#-equivalent pipeline (D168). The prose surface parses `value |> stage` left-associatively. The `Pipeline` node records `LastArgument` (default) or `FirstArgument`; F# is the compatibility case for the former and Elixir for the latter. Lowering desugars to `Call` with the value appended or prepended, and the portable expression document preserves the node and style. A stage may carry whitespace arguments (`x |> f a`); general partial application and currying outside a pipeline are deferred.

Reverse pipe (D171). `f <| x` is supported as `ReversePipeline` for F# package compatibility, with `f a <| x` appending the value and right associativity for chains. It is discouraged in new Scale code; the forward pipeline or a direct call is preferred. The portable document preserves the node, and lowering appends the value to the left call.

Explicit partial application (D170). A call argument may be `_`; `subtract(_ 10)` becomes a one-hole closure. Supplied arguments are evaluated once at creation, holes bind when applied, and a call with no holes remains `Call`. This is explicit partial application, not automatic currying. The portable expression document preserves `Partial`; lowering creates a lambda with hole parameters and captured supplied values. `apply` handles the resulting kernel closure.

Text side channels removed (D167). P3d is implemented. The legacy text quotation reader, `toTokens`, and the token-vector history are gone. `EvaluatorCode.decode` accepts only `[QuoteTag; documentId]`; a text tensor is not code. History stores code values in its current-term and program fields. Surface clause grammars and the kernel bridge keep their token projections as parser/bridge boundaries; the kernel term loop is later work.

These Struct/IsByRefLike annotations are host performance choices and reading examples, not semantic guarantees: they do not create ownership, lifetime, aliasing, or promotion rules.

Evaluation-error taxonomy (D94). `EvaluationError` is one 41-case union whose related faults are typed families (`UnknownKind`, `ErrorSubject`, `ExpectedValue`, `ArityFault`, `ChannelFault`, `SortFault`, `LimitFault`, `EnumFault`, `ActorFault`). Family cases are written qualified to avoid union-case shadowing. `describeError` and `Violations.ofEvaluationError` keep every previous message and rule string, so the taxonomy changes representation only; `types.fs` splitting remains a later step.

Three-layer kernel boundary (D95). The kernel is the abstract machine; dispatch, classes, unification, set algebra, ownership, guards, conditions, and supervision policy are Prelude code, and host adapters contain no Scale semantics. The boundary supersedes `docs/kernel-inventory.typ` where the two conflict. `components/kernel/scale_abi.h` and `components/kernel/Scale.Kernel.fsproj` define ABI version 2 with nine exports: `scale_init` installs callbacks and validates the image, `scale_set_callbacks` replaces the table for a live handle, `scale_run` executes an exported unit, `scale_verify` verifies a term-program document and writes its receipt, `scale_image` imports or exports the retained image, `scale_host_call` forwards a named package/function call, and `scale_transport_send` forwards a framed target/payload to the host. The typed D94 errors are flattened to `(code, message)` only at the ABI edge. The kernel assembly compiles `types.fs`, `tensor.fs`, `index-transformation.fs`, `lexicon.fs`, `store.fs`, `scalar-read.fs`, `stack.fs`, `lowered-ir.fs`, `term-ir.fs`, `term-portable.fs`, `term-verification.fs`, `kernel-state.fs`, `lowered-abstraction.fs`, `prelude-image.fs`, `primitive-wire.fs`, `prelude-execution.fs`, and `abi.fs`. The Prelude image path exists, but the shipped Prelude is still the F# reference implementation; `scale_run` executes an image unit in the kernel rather than forwarding the whole program.

Pattern matching (D96). The runtime pattern IR and the ground-subject matcher are landed in `components/prelude/evaluator/pattern.fs` with `Fresh`, `Pin`, `Repeated`, and `Shadowing` roles and the `match` surface. `PatternFault` names `NoClauseMatched`, `MalformedPattern`, `MissingPin`, `UnboundRepeated`, and `PatternUndecided`. The IR is `pattern-ast.fs` and the closed portable form is `pattern-portable.fs`, a versioned JSON document with canonical kinds and pinned predicate versions; an open subject is matched through the unifier and a `when` guard runs a named predicate. `PatternAnalysis` adds best-effort reachability, redundancy, and closed-kind exhaustiveness. Templates remain open.

General IR (D97). `docs/general-ir.typ` defines one portable IR for expressions, patterns, packages, kinds, predicates, sorts, conditions, dispatch, and actor messages, with the six-function adapter contract and pinned versioning. The locked contract is a partial portable core with host-marked extensions, canonical JSON v1, a ground decision tree plus an open-subject unifier, semantic

type references with optional non-semantic hints, package-local content-hashed registries with exact pins, default-deny capabilities, unit-scoped source maps, and a portable-only prelude bootstrapped on F# first. The pattern IR and closed portable pattern document are Phase 1 and are landed; the general expression AST (`kernel/expression-ast.fs`) and its canonical JSON v1 document (`kernel/expression-portable.fs`) and the package/unit document (`kernel/package-ast.fs`, `kernel/package-portable.fs`) are landed.

Kernel boundary correction (D98). The portable pattern, expression, and package IR plus pattern analysis, history, quotation-run, and the common and stack evaluator slices are now Prelude code. The lowered abstraction reads no ownership state; cache guards are validated by a Prelude callback. `runtime-types.fs` carries the actor/channel/scheduler/ownership/supervision/ monitor/method types, `EvaluationState`, and the full `EvaluationError`; `epoch.fs` and channel store encoding moved to the Prelude; the lowered IR/abstraction are kernel code (D99). The kernel holds core types, tensor/store primitives, the lowered IR/abstraction, image loader/execution, primitive wire, and the ABI; a boundary fitness check asserts the forbidden types are absent. `scale_init` validates and retains the prelude image and also delivers it to the host adapter during the transition; `scale_run` resolves an exported unit and executes it with the kernel abstraction.

Current status (D116)

This is the live status. Decision records keep their historical numbers.

- Suite: **1530 checks** (1488 examples and 42 properties).
- Demo content root (D141): `Demo.demoDirectory` resolves to `components/demo/demo` in a checkout and to `<base>/demo` in a publish payload; the suite reads its generated baselines through the same resolver.
- Prose surface (D142–D149): `docs/prose-surface.typ` records the accepted design (lowercase keywords, `.prose` files, the comma as the multiple-choice delimiter, spaced vocabulary names and free `of` phrases, text with spaces, `close`-delimited nested matches, and a sentence sequence that becomes a `Block`). Both the inline form (`match e, when p then e, otherwise e.`) and the line form parse to `ExpressionAst.PatternMatch`, with named `ProseError` refusals and thirty-three parse/equivalence checks. The sequence lowers with `EvaluatorLowering` and runs on the kernel `LoweredAbstraction`; the final stack top is the result. `scale --prose <path>` reads and runs a `.prose` file (D149); the `run` and `capture` shell effects are Stage 1a host words executed by `components/shell-host` (D183). Books and spells are Stage A (D184): a `.book` file compiles to a `PackageAst` package, `main` makes it a spell, `scale --book` and `scale --cast` run or call it, and prose `cast` calls it directly. Prose refuses `logic`. The `with` connective is implemented (D187); the REPL is not implemented.
- Granular suite: **33 cases** (11 snapshots, 13 integrations, and 9 JIT-only writer checks); the duplicated example/property cases were removed in D114.
- Test batch (D148): P1 compares the interpreter and the lowered abstraction over generated arithmetic; P2 compares the inline and line prose forms; S1 pins the lowered differential corpus as `snapshots/lowered-corpus.verified.txt`. Discovery is 91 tests.
- Kernel: **2665 lines / 2003 code**.
- Prelude core (excluding tests, tools, demo/acceptance, and edge/verification): **15300 lines / 11851 code**.
- Edge/verification (not part of the initial port): **5818 lines / 4431 code**.
- Demo and acceptance (not counted as Prelude code): **1565 lines / 1131 code**.
- ABI: nine exports; `scale_init`, `scale_set_callbacks`, `scale_run`, `scale_verify`, `scale_image`, `scale_host_call`, and `scale_transport_send` are implemented.

- Expression lowering (D119) is total over the closed constructor universe. A supported node lowers; a declined node is a named `LoweringRefusal`, never a hidden run-time `Fail`. `call` over a word target is lowered and agrees with the token interpreter.
- Lexical bindings (D122) resolve through the frame chain and slots. Lowered `let` writes both a frame binding and a fresh slot; a bare word resolves to the nearest binding, `lookup` still reads the frame, and nested lets shadow. Reserved binding names and calls to bound names are named refusals.
- Lambda parameters (D123) bind to slots in a named unit. `MakeClosure` carries the arity, `CallClosure` refuses an underflow by name, `lowerExpression` returns a `LoweredProgram` with a unit table, and `LoweredAbstraction.runProgram` resolves units. Enclosing lexical names are captured conservatively.
- A shared differential corpus (D124) runs eighteen token programs through the interpreter and the lowered abstraction, comparing scalar results and failure families. It pins the token-lowering spellings to the interpreter's `lessThan` and `greaterThan`, and it found and fixed the abstraction's opposite `rotate` direction.
- Pattern matching (D125) compiles to inspection, branches, and slots for ground subjects. Literals, wildcards, fresh variables, constructors, and typed patterns lower; pin, repeated, shadowing, and guards are named refusals. The kernel gains the generic `IsText` and `TensorEntry` primitives.
- The IL writer (D126) is a JIT-only backend behind `EnableWriter`. The writer harness (D128) replaces the raw-double prototype: the emitted method takes and returns the real `KernelState<EvaluationState>`, and calls the AOT-safe `WriterRuntime` for value instructions, stack access, dispatch, and failure. `LoweredAbstraction.step` is the shared single-instruction semantics.
- Compiled artifacts exclude history, bookmarks, and rollback (D120). `LoweredInstructions.historyScope` is `HistoryExcluded`, the closed instruction union has no step record, and a test fails if one is added without revisiting the decision. The interpreter remains the only history-recording engine.
- The F# Prelude remains the shipped reference; the Scale Prelude port is deferred until the core port is complete (D133).

Language specification (D129). `docs/specification.typ` is the normative language document. It covers the core and the extensions and records every feature as implemented, partial, planned, or refused, with the decision and the test as evidence. The topic documents remain the rationale and the deep dives.

Open-question decisions (D130–D132). The alpha reload is SageFs `rebuild=true` (the only MSBuild path; shell MSBuild stays banned). The writer next emits one method per unit and adds closure, tensor, and pattern opcodes. `Pin`, `Repeated`, `Shadowing`, guards, and the declaration/message nodes stay interpreter-only. The C host adapter and conformance harness landed (D136). The surface has no comments and one-word text literals; whitespace separates words. `set-subset` is the value-level subset. The Prose sentence grammar and Z3 verification are deferred until a product or language need. Sequences and recursive types are target; multisets and co-products wait for a consumer.

Open-question decisions (D133). Lambda capture is free-variable (only the enclosing names the body uses). The Scale Prelude port is deferred until the core port is complete. Implementation-instance IPC is a target track, planned (D151). Status-matrix ratchets follow the next writer unit slice. D122's lexical `let` is final.

Writer units (D134). The writer emits one method per `LoweredUnit`. A `MakeClosure` immediately followed by `CallClosure` compiles to a direct call to the unit method; `EnterUnit`/`ExitUnit` manage

the frame and `PushCapture` is supported. A first-class closure remains a named refusal. The granular suite is
26.

Closed-decision ratchets (D135). Eight checks pin the closed rows: no comments and one-word text literals (`splitText`), `set-subset` true and false, `subtype` unresolved, and the interpreter's `Pin` match/mismatch and `Shadowing`. The example suite is 1298; the granular suite stays at 26.

C adapter and conformance (D136). `components/adapter-c/` loads the kernel, resolves the nine exports, and checks `scale_abi_version()` against its compile-time `SCALE_EXPECTED_ABI_VERSION` before any other call; a mismatch is refused. `components/adapter-c/conformance/conformance.c` checks both sides of version equality, the prelude dispatch, a declared primitive, the result, and handle teardown. The compile check, the fake-kernel version probe, and the full native run are alpha gates: `just kernel-aot` publishes the NativeAOT kernel and runs the harness (D180). The example suite is 1412; the granular suite is 33 cases.

Polylith workspace (D137). `workspace.edn` lists the nodes; the `.fsproj` graph is the dependency truth; `just poly-check` enforces components → components, bases → components, projects → components/bases, cycles, and interfaces. Stage 1 moved the kernel to `components/kernel` and the C adapter, with its conformance harness, to `components/adapter-c`; stage 2a moved the app to `bases/scale-cli` and added `projects/scale` as the publish target. Stage 2b splits the libraries. Nu joins as its own node, wrapped by `nu-host`; both are NativeAOT, so the C ABI is the seam. D181 records the host and Gaia-on-ImSim plan; no Nu base, project, or build lane is integrated yet. D182 records the colocated jj workspaces: `Scale-code` is the sparse Scale-only checkout, and the main workspace keeps Nu. D138 records the module dependency map; D139 lands the full split: `core`, `typing`, `prelude`, `verification`, `demo`, and `cli` are components, `development/scale` holds the suite, and the base is `entry.fs`. D183 adds `shell-host`, the process effect boundary for the prose shell; D184 adds `book-host`, the book/spell loader and prose `cast` adapter. The example suite is 1530; the granular suite is 33 cases.